

Tab 1

- Web App (React + TS backend)
- Lot of AI-developed capabilities
- Non-monolithic
- Working prototype - trend analysis (watches through Tiktok and picks up on trends - for example, when Charlie Kirk got shot, it updated a graph since the topic became frequent on TikTok - useful for brands).
- LLMs - The core of the project is that the LLM speaks to you based on the various metrics listed above. Context is personalized to brand identity and trends.
- Possibly LangChain? LLM would connect to other models and tools.
- Likely to run on python.
- Each user will spend 20 dollars per month for the subscription. The cost limit is 20 bucks for us.
- Working foundation of the app exists.
- One of the features to implement is: If a user uploads a video on youtube, and if, for some reason, they swore in the first 30 seconds, the algorithm suppresses them (note that the rest of the video doesn't matter). A lot of people may not catch that at the very start.
- Our app will automatically upload the video on a schedule, it will review the video for checks like removing swear words.
- Selling point is that some of these rules are difficult to remember.

Notes:

- Microservice-style architecture
- Database, auth, frontend, backend, adapters for each social media (insta, tiktok, pinterest, facebook, youtube, possibly threads. Linkedin and twitter have paid apis)
- Database type - mysql + postgres + redis clusters + s3
- Database hosting service - Railway
- Drizzle ORM
- Auth - WorkOS - IdP (different teams and organizations, so users have to authorized based off of that)
- LangGraph is already used (look into it, not implemented much)
- No streaming implemented (no need for realtime event processing)
- Event handler - BullMQ
- Domain name - xclix.ai - landing site (not actual app - static page)
- Website hosting - Bluehost
-

Tech Stack

React - Frontend

TS Backend - Backend

MySQL - Legacy DB

Postgres - Current DB

Redis - Caching

S3 - Object DB

Railway - DB Hosting Service

Drizzle - ORM

WorkOS - Authentication/Authorization

LangGraph - Chaining Models

BullMQ - Event Processor

Desired Additions:

Vector Database (MongoDB?)

For a microservice-style architecture, we'll want:

- A dedicated database for each service (service listings will have to be written out)
- A saga pattern for data consistency. I believe that this will be done best with an orchestration approach rather than a choreography approach. So, we'll use BullMQ as a queueing service, and we'll have to figure out an orchestrator. If that turns out to be expensive, we could use the Pub/Sub method.
- We'll want an API Gateway pattern (basically something that will act as a reverse proxy to a group of microservices for clients). It also improves security by providing at most a single entry point.
- We'll also want an aggregator design pattern, which collects pieces of data from various microservices and returns an aggregation of all of them.
- We might want a circuit breaker design pattern (so that we can cut off communication with a microservice that might be experiencing high latency or unresponsiveness). Basically, you need to define failure conditions, and then have an object to determine whenever failure conditions are met.
- You might also want a command query responsibility segregation (CQRS) to reduce risks associated with multiple services accessing or writing to the same data at the same time.
- In certain service-service architectures, we could incorporate asynchronous messaging, to prevent services from waiting on responses when this is unnecessary. You can also use a message queue to encourage decoupling between microservices (which should honestly already be implemented between frontend and backend services).
- Event sourcing, which is used when you want to capture all changes in an entity's state. Event stores like Kafka can keep track of event changes and can function like a message broker, which helps with the communication between different microservices.

User Functionality Requirements:

- Allows users to connect all of their different social media accounts in one place.
- Allows users to upload videos to multiple platforms at once.
- Allows users to schedule the upload of videos.
- Automatically processes videos to detect possible demonetization or contra-algorithm (such as swearing within the first 60 seconds) risks.
- Automatically changing description + adding relevant tags based on current trends.

Trend Analytics Requirements:

- Provides trend analytics to determine popular niches, tags, or video concepts.
- Provides LLM to generate video ideas, with context personalized to brand image
- Sentiment analysis on comments of previous posted videos + querying capabilities to locate certain characteristics of comments.

Possible Additional Services:

Video Services

- Provide video analysis for determining what sections of various videos perform most poorly.
- Automatic video reuse (transform most-viewed portions of long-form videos into shorts and upload them automatically).
- Automatic video resizing based on platform
- Hook analysis and recommendations (determining how effective the hook is for keeping the user for the first 30-60 seconds of a video).

Audio Services

- Background music analysis and suggestions

Thumbnail Services

- Object and image analysis on thumbnails to determine various popular features (like head/face positioning, color scheme, etc).

Deployment Services

- Automated A/B testing and analysis + adjustment recommendations (like what features should be adjusted for A/B testing).
- Auto-replying to common questions sent in the comments
- Notifications when trends align with brand image/user's niche
- Automated captions and localization

Primary Requirements:

- Allows users to connect all of their

Future Tech Stack

Vector Databases:

- Look into Chroma vs Pinecone vs MongoDB

Object Storage:

- S3

Relational Databases:

- Postgres

Fast Caching + KV Database:

- Redis

Should we use a Document Database??????????

Graph Database

- Neo4j

Data ETL pipeline for training models:

Platform Managment

Platforms to include in **MVP++**

- TikTok
- YouTube
- Instagram
- LinkedIn
- X

This list is enough for the initial product scope, but the platform layer should be designed so additional social networks can be added later without rewriting core application logic.

Platforms should be managed through a shared integration framework. Each social platform should be represented as a provider with a stable internal key (slug), display metadata, authentication configuration, supported scopes, connection status, and a set of declared capabilities. The application should not treat every platform as a completely separate feature. Instead, it should use common concepts such as providers, connections, resources, credentials, capabilities, and metrics.

A provider represents the platform itself, such as YouTube or LinkedIn. A connection represents a brand's authenticated relationship with that provider. A resource represents the actual account or object the brand can manage on that platform, such as a YouTube channel, Instagram business profile, LinkedIn page, TikTok account, or X profile. Keeping these separate allows one platform connection to expose one or more manageable resources, and it gives the product a consistent model across platforms.

Provider-specific behavior should live behind platform adapters. Each adapter should be responsible for translating the application's normalized operations into the external platform's API calls. Common operations would include starting OAuth, completing OAuth, refreshing tokens, discovering resources, fetching analytics, publishing content, and handling webhooks. The rest of the application should call these operations through a common orchestration layer instead of calling platform APIs directly.

Each platform should declare which capabilities it supports. For example, one platform may support analytics but not publishing, while another may support publishing but only for certain account types or scopes. Capability checks should happen before an operation runs so the product can give a clear response when something is unsupported or unavailable. This also gives the frontend a reliable way to know which actions to show for each connected platform. Credential management should be centralized. OAuth access tokens and refresh tokens should be stored securely, refreshed automatically when possible, and exposed to platform adapters only through a controlled token lookup flow. Platform adapters should not own long-term credential storage. Their job should be to perform the provider-specific token exchange or refresh when required.

Analytics should also be normalized. The application should use shared metric names such as views, impressions, likes, comments, shares, followers, subscribers, watch time, and retention where possible. Each platform adapter can map those normalized metric names to the provider's API-specific metric names. This lets the product present analytics consistently even though each social API structures data differently.

Adding a new platform in the future should generally involve:

1. Defining the provider and its configuration.

2. Declaring its supported capabilities and required scopes.
3. Implementing the provider adapter.
4. Mapping platform resources into the application's resource model.
5. Mapping platform analytics into normalized metrics.
6. Registering the provider so the rest of the application can discover and use it.

5/19/26 Meeting Outline

Meeting Notes - 5/19/2026 11:00 AM

Outline:

Current State of The Project:

- What the project already has working.
- What the project already has partially working.
- What the project has not working.
- Things that need to be changed.
- Things that don't need to be changed now, but later.
- Project Walkthrough:
 - Frontend

AI Development Policy:

- Backend
- What do we plan on using AI for?
- What do we think AI can do best?
- What should we NOT use AI for?

Desired Tasks:

- Backend vs Frontend
- Languages
- None technical tasks

Project Feasibility:

- Timeframe
- Our projected availability
- Technical possibility of our desired features

Feature Priority:

- Foundational features we must implement first.
- How we will begin implementing them.

Code Refactoring:

- Things that need to be changed
- Things don't need to be changed now, but later.

Misc:

- Canva Genesis Notes and recent additions

AI Development Policy:

- Use:

- Documentation
- Refactoring
- Tedious tasks

- Don't use:

- Features we don't yet fully understand
- Unsupervised design and implementation

Switch to PNPM

Delete useless legacy code

Set up documentation platform and standard

Research into using GO + TS backends

Tab 6

Documentation:

Backend:

Root directory: Config files for drizzle, [TODO.md](#), and [ARCHITECTURE.md](#) (ill look at this later)

Src:

- App:
 - Config
 - [ctx.logger.ts](#) - Creating a pino logger and making it an export so all files can use it.
 - [di.ts](#) - Dependency injection config even tho its not setup yet?
 - [env.ts](#) - Zod setup for env vars (although I did notice that the regex for port only specifies that it should be a number buuuuuuuut it should technically only be a number greater than a 1000 and less than 65535; idt it can be less than 1000 since those ports are reserved but not sure). It may break if someone puts in something like 100000? You could probablyyy do smth like this: `/^[1-5]\d\d\d\d$/` to limit it to any number between 10000 and 59999 to leave it as regex without actual validation cuz i'm not sure if you can do that with zod. I'll check it out later.
 - [logger.ts](#) - pino config, just sets the number to debug or info and removes some paths.
 - HTTP
 - Middleware - looks like it contains authentication (idk why authguard calls it authorization, it doesn't seem to be defining permission for anything, it just checks if you have a valid session), a util func for error handlin, a rate limiter (for redis?), a util func to log requests with pino, and authorization (via requirePerm).
 - [Routes.ts](#) - Changing the path for api request handlers (probably so its less messy) - The types are listed as "any" which should probably be changed.
 - [Server.ts](#) - setup for CORS, sets up queue service, sets up the web server, and a websocket server (?) - Kinda need a bit of an explanation for whats going on here.
 - [Swagger.ts](#) - sets up swagger
 - [Webhook.ts](#) - also self explanatory - listen to /webhooks endpt for requests from workos
 - SMTP - sending emails (i'll look into this one later)
 - Jobs - BullMQ setup + cron job
 - Events - only one file, [bus.ts](#) and it just creates an event bus (for pub/sub?)
 - Observability - Checkout later
 - Scripts - generic util scripts (?)
- Shared
 - DB - mysql and postgres stuff
 - Cache - redis

- Http - only one file (axiosManager) - sets up Axios client (idk the point of axios tbh)
- Files - only one file [docling.ts](#) - turns files into json
- Types - self explanatory
- Utils - self explanatory
- Resources/instructions - prompts?
- Modules - large folder, ill get into this one later, but i will list out the file structure:
 - Auth
 - Api
 - Domain
 - Infra
 - Tests
 - Brands
 - Api
 - Domain
 - Infra
 - Integrations
 - Api
 - Core
 - Domain
 - Infra
 - Lib
 - Orchestrator
 - Policy
 - Providers
 - tokenManager
 - Tests
 - Scheduler
 - Api
 - Domain
 - Infra
 - Inbox
 - Api
 - Domain
 - Infra
 - Analytics
 - Api
 - Domain
 - Infra
 - Ai
 - Api
 - Domain
 - infra

Tab 7

Necessary features:

- Uploaded video storage
- Another worker proc for specifically LLM-based jobs (bullMQ stuff)
- API gateway for LLMs

Services we want to provide:

1. Connect all social media accounts in one place
2. Schedule video uploads
 - a. Need to modify [schedule.service.ts](#) i think
3. Demonetization thing
 - a. Get a transcript of the video that was uploaded
 - b. Handler that checks transcript for profanity
 - c. Add indicators to the frontend
4. Generate description + tags
 - a. Pull information from a trends table (what kind of info?)
 - b. Get LLM to generate description based on transcript, current trend data, personal brand data
5. Trend analytics
 - a. We need to define what we're going to collect from social media to identify trends
 - b. Where can we pull data from and what kind of data can we pull?
 - c. We can store this information within tables in postgres
 - d. How frequently do we pull the data? How often do trends change?
6. LL video idea generator
 - a. Should pull information from trends, brand image, possibly best performing videos
 - b. Generate 10 ideas?
7. Video Analysis
 - a. lirc yt's api gives audience retention at certain intervals. Idk about other platforms though. We need to look into that pai

LLM Service Essentials

LLM Service Essentials

Major Features

1. Prompt Playground

The Prompt Playground is the main testing interface for prompt development. It should allow us to compare and contrast prompts to refine the prompts used in the main application.

Features:

- Create prompt experiments
- Save prompt versions
- Compare prompt A vs prompt B
- Compare multiple models on the same prompt
- Compare different temperatures/settings
- Run the same input repeatedly
- View outputs side by side
- Score outputs manually
- Add notes to each prompt version
- Mark one prompt as "best current version"

2. Model Comparison

The service should support testing the same prompt across multiple models. This allows us to test which models are best for different tasks. For example, gpt 5.5 for human-like chatting and gemini 3.5 for content ideation.

Features:

- Run one prompt across multiple models
- Compare output quality
- Compare speed
- Compare token usage
- Compare estimated cost
- Compare formatting accuracy
- Compare consistency

Models:

- OpenAI Models
- Anthropic Models
- Google Models
- Locally Hosted Models

3. Prompt Template System

The service should support reusable prompt templates. A prompt should not be hardcoded into one route. It should be editable, versioned, tested, and reused.

Example:

...

You are an AI assistant for a social media management platform.

Brand:

{{brandName}}

Platform:

{{platform}}

Tone:

{{tone}}

Task:

Generate a message for the following goal:

{{goal}}

...

Features:

- Variable injection
- Required variable validation
- Optional variables
- Default values
- Prompt preview before execution
- Prompt version history

4. Run History

Every LLM call should be saved as a run.

To store:

- run_id
- prompt_id
- prompt_version
- full rendered prompt
- model used
- input variables
- output
- latency
- token usage

- estimated cost
- tool calls used
- errors, if any
- timestamp
- evaluator score
- Notes

5. Tool Calling System

The service should support tools that the LLM can call during a workflow.

Tools are external abilities given to the model. Instead of only generating text from memory, the model can use tools to retrieve information, analyze content, or perform structured actions.

For testing, tools can be mocked, real, or partially real.

6. Agent Workflow Builder

The service should eventually support multi-step workflows. This is where we use LangGraph.

Example to generate video-based post:

1. Lookup YouTube video.
2. Get transcript.
3. Summarize video.
4. Extract key points.
5. Generate social media post.
6. Critique the post.
7. Rewrite final version.

Example with LangGraph:

Input:

YouTube URL

Graph:

Video_lookup -> transcript_fetch -> video_summary -> content_idea_generation ->
draft_generation -> critique -> final_output

7. Prompt Testing Dataset

The service should have reusable test cases. This lets us test prompt changes against old examples.

8. Regression Testing

Regression testing means checking that a new prompt version does not become worse than an older version.

Features:

- Run old test cases against new prompts
- Compare new output to old output
- Score both outputs
- Flag major drops in quality

9. Regression Testing

The service should track how much each experiment costs.

Must track:

- input tokens
- output tokens
- total tokens
- estimated cost
- model used
- tool calls used
- run duration

10. Caching

We don't have to include this now, but it would be beneficial later down the road. Could be used for tool calls.

Required Tools

1. Web Research Tool

The Web Research Tool allows the LLM service to search the web and gather current information.

Use cases:

- Research a trending topic
- Find current news about a brand
- Gather information before generating content
- Check facts before writing
- Summarize search results

Useful tools found by Chat:

Tool category	What it does for Xclix	Existing maintained option	My recommendation
Web search	Research trends, news, creator topics, platform updates	Tavily MCP, Firecrawl MCP	Use Tavily for search, Firecrawl for scraping/crawling
Web scraping	Extract clean text from articles, blogs, docs, competitor sites	Firecrawl MCP, Apify MCP	Use Firecrawl for clean web pages, Apify for difficult/social scraping
Social/public data scraping	Find examples, competitor posts, public content patterns	Apify MCP	Use for testing, be careful with platform ToS
YouTube metadata	Look up videos, titles, channels, descriptions, dates	YouTube Data API	Use official API directly
YouTube transcript	Turn videos into text for analysis	youtube-transcript-api , YouTube transcript MCPs, Supadata	Use Supadata or your own wrapper, transcript MCPs are fine for testing

Video analysis	Summarize videos, find clips, extract hooks	Supadata, custom pipeline with transcripts and frames	Start transcript-only, add visual analysis later
Instagram publishing	Publish posts, reels, carousels	Instagram Graph API	Build official API integration yourself
TikTok publishing	Upload/post content	TikTok Content Posting API	Build official API integration yourself
LinkedIn publishing/analytics	B2B posts and campaign insights	LinkedIn Marketing APIs	Build official integration yourself
Threads publishing	Manage Threads presence	Threads API	Build official integration yourself
Pinterest publishing/analytics	Pins, boards, account analytics	Pinterest API v5	Build official integration yourself
X/Twitter posting	Create posts, read trends/data depending on access	X API v2	Build official integration carefully because pricing/rules change
Internal docs/context	Let AI read your docs, specs, notes	Notion MCP, Google MCP services	Use official hosted MCPs where possible
Dev workflow	AI can inspect repos, issues, PRs	GitHub official MCP	Use it internally, great for engineering automation
Team communication	Search Slack, summarize discussions, send updates	Slack official MCP	Useful internally, not needed for product MVP
Database inspection	Let AI inspect schema/test data	Postgres MCP options	Use only in local/dev, never broad prod write access
Browser automation	Test workflows, inspect pages, automate UI checks	Playwright MCP	Useful for QA/testing, not core product

File/storage search	Search uploaded PDFs/images/assets	Custom tool around S3/R2/GCS	Build your own because app permissions matter
Prompt/eval tooling	Compare prompts, score outputs, regression test	LangSmith, Braintrust, Promptfoo	Use one early, or build a small internal eval runner

We should probably use Tavily for searching

We should try out Supadata for video analysis and transcription instead of implementing our own if we don't have to.

Let's also look into Apify and Ensembledata to see if its more cost effective than running our own scrapers (I doubt it but saves us a lot of development)

Phyllo is another tool that has a lot of things we would have to implement our self

Tab 9

Full List of Features for the LLM Service:

Saving Prompts and Prompt Comparisons:

Prompt Object: {

- Created_At: DateTime
- Updated_At: DateTime
- Name: string
- Prompt_Id: UUID string
- Project_Id: UUID string
- Best_Version_Id: UUID string | NULL (for new prompts)
- Description: string | NULL
- Compound Unique Constraint [Project_Id + Name]

}

Version Object: {

- Created_At: DateTime
- Prompt_Id: UUID string
- Id: UUID string
- Name: string
- Default_Temperature: float
- Default_Top_P: float
- Default_Max_Tokens: integer
- Version Messages Array: [type BaseMessage (from langgraph_core.messages)]
 - Note: all images and documents will be referenced via image_url and file_url respectively, with the url property linking to an S3 URI, validated via boto3
- Bound_Tools Array: [Tool_Id: UUID string] (make sure this is type String not Tool to prevent an extra join)
- Response_Schema: JSON
- Context_Schema: JSON
- Description: string | NULL
- Compound Unique Constraint [Prompt_Id + Name]

}

Run Object: {

- Id: UUID string
- Workflow_Run_Id: UUID string | NULL (if not part of a workflow)
- Prompt_Id: UUID string | NULL (necessary if the prompt isn't saved immediately)
- Version_Id: UUID string | NULL (same reason as above)
- Project_Id: UUID string (can't be NULL as runs have to be associated with a project)
- Executed_At: DateTime
- Model_Company: ProviderType Enum [OpenAI, Anthropic, Google, Custom]
- Model_Name: string
- Model_Output: JSON (in case the model returns tool output or images/files)
- Input_Variables: JSON object ({variable_name: variable_value} for injection)
- Output_Quality: JSON
- Speed: integer (in milliseconds)

- Input_Tokens: integer
- Output_Tokens: integer
- Estimated Cost: float
- Formatting Accuracy: float
- Temperature: float (this is for possible changes during the run to the default temperature)
- Top_P: float (same as temp)
- Max_Tokens: integer (same as temp)
- Status: Enum (status, failed, timeout)
- Status_Message: string (error message or a 200 OK string)

}

ContextSchema example: {

- "Type": "object"
- "Properties": {
 - "Brand_name": { "type": "string" },
 - "Tone": { "type": "string" }
- },
- "Required": ["brand_name", "tone"]

}

In order to use that effectively, use {brand_name} and {tone} in the messages array.

Canvas objects:

Workflow Object (Canvas object): {

- Group: string
- Name: string
- Id: UUID string
- Project_Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Compound Unique Constraint [Project_Id + Name]

}

Workflow_Node: {

- Workflow_Id: UUID string
- Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Type: Enum [Prompt, Tool, Custom]
- X-coordinate: integer
- Y-coordinate: integer
- Configuration: JSON (can be prompt_version_id, tool_id, tool_group_id, custom_script_id)
- Name: string (does not have to be unique)
- Description: string | NULL

}

Custom_Script (currently all routers need to be custom): {

- Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Name: string
- Code: Text (sent to an isolated environment to run - Docker container/WASM/Google Cloud Func/Lambda)
- Expected_State_Schema: JSON
- Project_Id: UUID string
- Compound Unique Constraint [Project_Id + Name]

}

Workflow_Edge: {

- Workflow_Id: UUID string
- Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Type: ENUM (loop, conditional, normal)
- Routing_Key: string | Null (multiple "edge" objects for each conditional edge, and routing key is the key in the json for a conditional edge)
- Source_Node_Id: UUID string
- Target_Node_Id: UUID string

- Name: string (does not need to be unique)
- Description: string | NULL

}

Workflow_Run: {

- Id: UUID string
- Workflow_Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Status: Enum [Pending, Running, Completed, Completed_With_Errors, Failed]
- Input_Variables: JSON (initial langgraph state class)
- Final_Output: JSON (final state class)
- Execution_Time_ms: integer
- Total_Cost: float

Testing Objects:

Dataset: {

- Name: string
- Id: UUID string
- Project_Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Description: string | NULL
- Compound Unique Constraint [Project_Id + Name]

}

Test_Case: {

- Name: string
- Id: UUID string
- Dataset_Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Description: string | NULL
- Input_Variables: JSON
- Expected_Output: string
- Assertion_Rules: JSON
- Compound Unique Constraint [Dataset_Id + Name]

}

Evaluation_Run: {

- Id: UUID string
- Dataset_Id: UUID string
- Version_Id: UUID string | NULL (for testing prompt versions)
- Workflow_Id: UUID string | NULL (for testing workflows)
- Project_Id: UUID string (for loading analysis per project)
- Created_At: DateTime
- Overall_Score: number
- Total_Cost: float
- Pass_Rate: float
- Status: Enum [Pending, Running, Completed, Completed_With_Errors, Failed]
- Updated_At: DateTime (for checking status)

}

Evaluation_Result: {

- Id: UUID string
- Evaluation_Run_Id: UUID string
- Test_Case_Id: UUID string
- Executed_At: DateTime
- Passed: boolean
- Output: JSON | NULL (what the model returned)
- Latency_ms: integer
- Input_Tokens: integer

- Output_Tokens: integer
- Error_Logs: String | NULL
- Cost: float
- Score: float | NULL
- Assertion_Snapshot: JSON (in case the test case gets changed)
- Expected_Output_Snapshot: string (in case the test case gets changed)

}

Tools:

Tool_Group: {

- Name: string
- Id: UUID string
- Project_Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Description: string | NULL
- Compound Unique Constraint [Project_Id + Name]

}

Tool: {

- Name: string (regex to make sure it has only numbers, letters, or underscores, and nothing else)
- Id: UUID string
- Project_Id: UUID string | NULL
- Tool_Group_Id: UUID string | NULL (if global tool, make project + group null)
- Created_At: DateTime
- Updated_At: DateTime
- Description: string (can't be null since the LLM reads the docstring to understand what the tool does)
- Execution_Type: Enum [Mock, HTTP, Langchain_Native]
- Execution_Config: JSON (contains a lot of information, but its specific to execution type, so to avoid polymorphic association, class table inheritance, and to avoid a bunch of Nullable fields, I think this is the best solution. Should also have a version tag to avoid invalidating old tools)
- Payload_Schema: JSON
- Response_Mapping: JSON | NULL (in case you get a huge response that needs to be filtered down by the backend)
- Active: boolean

}

Since there are no explicit references to tool_id inside of the Workflow Node config, you don't want to delete a tool that's in use by a node. So just add checks in the backend (since I don't want to have a bunch of Nullable values in the config)

Projects:

Project: {

- Name: string
- Id: UUID string
- Description: string | NULL
- Created_At: DateTime
- Updated_At: DateTime

}

Project_Member: {

- Id: UUID string
- Project_Id: UUID string
- User_Id: UUID string
- Created_At: DateTime
- Updated_At: DateTime
- Role: Enum [Owner, Admin, Regular, Viewer]
- Compound Unique constraint [Project_Id + User_Id]

}

User: {

- Id: UUID string
- Name: string
- Email: string
- External_Auth_Id: string (webhook to WorkOS)
- Created_At: DateTime
- Updated_At: DateTime
- Unique constraint: External_Auth_Id, Email

}

Provider_Credentials: {

- Id: UUID string
- Project_Id: UUID string
- Added_By_Id: UUID string | NULL (to prevent cascades from deleting provider credentials)
- Name: string (in case we want multiple API keys per provider for different use cases/fallbacks)
- Description: string
- Provider_Name: ProviderType Enum [OpenAI, Anthropic, Google, Custom]
- Base_URL: string | NULL (for custom providers)
- Encrypted_API_Key: string
- Created_At: DateTime
- Updated_At: DateTime
- Compound Unique Constraint [Project_Id + Name]

}